AI Web Translator

软件架构设计文档

版本 1.0.0

2026年3月

目 录

1. 概述

2. 系统架构

3. 核心模块设计

4. 数据流与交互

5. 关键技术实现

6. 跨平台适配

7. 部署与构建

1. 概述

1.1 项目简介

AI Web Translator 是一款基于 AI 大模型的智能网页翻译浏览器扩展，支持 Chrome 和 Safari 双平台。该扩展采用先进的语义理解技术，实现高质量的英中翻译，并提供独创的双向词汇高亮对应功能。

1.2 核心特性

- **AI 驱动翻译**：基于 Kimi AI 大模型，翻译质量高，理解上下文
- **双语对照显示**：保留英文原文，中文翻译显示在原文下方
- **双向词汇高亮**：鼠标悬停词汇时，AI 自动识别对应关系，原文与译文双向高亮
- **专业术语保留**：智能识别技术词汇、专有名词，保持不翻译
- **双平台支持**：同时支持 Chrome 和 Safari 浏览器

1.3 技术栈

层级	技术	说明
前端框架	原生 JavaScript	无框架依赖，轻量级实现
样式方案	原生 CSS	支持深色模式
AI 服务	OpenAI Compatible API	支持 Kimi、OpenAI、Azure 等
存储方案	chrome.storage / browser.storage	跨平台存储适配
构建工具	Xcode / 原生加载	Safari 需 Xcode 构建

2. 系统架构

2.1 整体架构

AI Web Translator 采用典型的浏览器扩展架构，由三个核心脚本组成：

- **Background Script（后台脚本）**：处理 API 调用、跨域请求
- **Content Script（内容脚本）**：注入网页，处理 DOM 操作和翻译展示
- **Popup（弹出窗口）**：用户界面，配置管理

2.2 架构层次

```
┌─────────────────────────────────────────────────────────────┐
│                          用户界面层                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │   Popup      │   │  控制栏      │   │  设置面板    │    │
│   │ (popup.html)│   │ (浮动按钮)  │   │ (配置页)    │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
├─────────────────────────────────────────────────────────────┤
│                          业务逻辑层                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │   Content    │   │   翻译引擎   │   │   高亮系统   │    │
│   │   Script     │   │  (AI驱动)   │   │  (双向)     │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
├─────────────────────────────────────────────────────────────┤
│                           服务层                             │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │ Background   │   │ API 适配器   │   │   存储管理   │    │
│   │   Script     │   │ (多Provider) │   │ (跨平台)    │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
├─────────────────────────────────────────────────────────────┤
│                          外部服务                            │
│                Kimi AI / OpenAI / Azure API                  │
└─────────────────────────────────────────────────────────────┘
```

2.3 项目结构

```
web-translator/
├── chrome/                                  # Chrome 扩展目录
│   ├── manifest.json                        # 扩展配置清单
│   ├── background.js                        # 后台脚本（API 调用）
│   ├── content.js                           # 内容脚本（DOM 操作）
│   ├── config.js                            # 配置管理模块
│   ├── api-adapter.js                       # API 适配器
│   ├── popup.html                           # 弹出窗口 UI
│   ├── popup.js                             # 弹出窗口逻辑
│   ├── translator.css                       # 翻译结果样式
│   └── icons/                               # 图标资源
│
├── safari/                                  # Safari 扩展目录
│   └── AI Web Translator/                   # Xcode 项目
│       ├── Shared (Extension)/              # 共享扩展代码
│       └── Shared (App)/                    # macOS/iOS App
│
└── scripts/                                 # 构建脚本
    ├── build-safari.sh
    └── sync-to-safari.sh
```

3. 核心模块设计

3.1 Background Script（后台脚本）

后台脚本作为扩展的服务端，主要负责处理与 AI API 的通信。

3.1.1 核心职责

- 接收来自 Content Script 的翻译请求
- 调用 AI API 进行翻译
- 支持多种 API Provider（OpenAI、Anthropic、Kimi）
- 处理 API 认证和错误

3.1.2 主要函数

函数名	功能	参数
handleTranslation	处理翻译请求	prompt, apiConfig
callOpenAICompatibleAPI	调用 OpenAI 兼容 API	prompt, config
callAnthropicAPI	调用 Anthropic API	prompt, config
testAPIConnection	测试 API 连接	apiConfig

3.2 Content Script（内容脚本）

内容脚本是扩展的核心，直接注入到网页中执行，负责页面文本提取、翻译结果显示和词汇高亮交互。

3.2.1 模块划分

```
Content Script 架构
├── 文本提取模块
│   ├── 元素过滤（跳过代码、脚本）
│   ├── 图片说明识别
│   └── 段落分块
│
├── 翻译处理模块
│   ├── 批量翻译队列
│   ├── 并行控制（10 线程）
│   └── 缓存管理
│
├── DOM 操作模块
│   ├── 翻译插入
│   ├── 原文/译文切换
│   └── 段落高亮
│
└── 词汇高亮模块（核心特性）
    ├── AI-Driven 映射表解析
    ├── 双向词汇匹配
    ├── 连续序列匹配算法
    └── 交互事件处理
```

3.2.2 词汇高亮系统

词汇高亮是本扩展的独家功能，采用 AI-Driven 映射方案：

数据结构

```
// 英文 -> 中文 映射（原始英文，非标准化）
wordMap: Map<原始英文, {
    original: string,      // 原始英文词组
    chinese: string[]      // 对应中文词组列表
}>

// 中文 -> 英文 反向映射
chineseToEnglish: Map<中文词组, 英文原文[]>
```

匹配算法

采用连续序列匹配算法，避免全局匹配导致的误高亮：

```
// 查找连续的单词序列
for (let i = 0; i <= wordElements.length - engWords.length; i++) {
    let isMatch = true;
    for (let j = 0; j < engWords.length; j++) {
        if (wordElements[i + j].dataset.word !== engWords[j]) {
            isMatch = false;
            break;
        }
    }
    if (isMatch) {
        // 高亮连续序列 wordElements[i] 到 [i + length - 1]
    }
}
```

3.3 Config Manager（配置管理）

配置管理模块负责处理多种配置来源，支持优先级覆盖。

3.3.1 配置优先级（从高到低）

1. 浏览器存储（用户设置）
2. window.openclaw（脚本注入）
3. window.KIMI_CODING_API_CONFIG（兼容格式）

4. localStorage（后备存储）
5. DEFAULT_CONFIG（默认配置）

3.3.2 默认配置结构

```
DEFAULT_CONFIG = {
  api: {
    baseUrl: '',
    apiKey: '',
    model: '',
    provider: 'openai-compatible'
  },
  prompt: '...',  // AI 翻译提示词
  display: {
    showOriginal: true,
    highlightTerms: true
  },
  behavior: {
    concurrency: 10,
    cacheResults: true
  }
}
```

4. 数据流与交互

4.1 翻译流程

```
用户点击翻译
    |
    ▼
┌──────────────────────────┐
|  1. 提取页面文本块
|     - 过滤不可翻译元素
|     - 识别图片说明
|     - 段落分块
└──────────┬───────────────┘
    |
    ▼
┌──────────────────────────┐
|  2. 构建 Prompt
|     - 页面上下文
|     - 待翻译文本
|       - AI-Driven 词汇表要求
└──────────┬───────────────┘
    |
    ▼
┌──────────────────────────┐
|  3. 并行翻译请求
|     - 10 线程并发
|     - 实时进度更新
└──────────┬───────────────┘
    |
    ▼
┌──────────────────────────┐
|  4. 解析翻译结果
|     - 提取中文译文
|     - 解析词汇对照表
|     - 构建 wordMap
└──────────┬───────────────┘
    |
    ▼
┌──────────────────────────┐
|  5. 插入 DOM
|     - 保留原文结构
|     - 译文显示在下方
|     - 添加词汇交互
└──────────────────────────┘
```

4.2 词汇高亮交互流程

```
用户悬停英文词汇
|
▼
查找wordMap
|
├── 遍历所有英文key
|   └── 拆分对比单词列表
|
└── 匹配成功
|
▼
获取对应中文词组
|
▼
在译文中查找并高亮
|
└── 连续序列匹配
```

4.3 存储方案对比

平台	API	用途说明
Chrome	chrome.storage.sync	配置持久化 跨设备同步
Safari	browser.storage.local	配置持久化 本地存储
Safari	localStorage	后备存储 降级方案
通用	Map（内存）	运行时数据 词汇映射表

5. 关键技术实现

5.1 AI-Driven 词汇映射

传统的规则映射方法依赖算法分词，准确率有限。本扩展采用 AI-Driven 方案：

提示词设计

要求 AI 返回结构化的词汇对照表：

```
| 序号 | 中文词组 | 对应英文原文 |
|-----|---------|------------|
| 1 | Amical | Amical |
| 2 | 的强大之处 | The power of |
| 3 | 上下文感知能力 | context awareness |
```

优势

- 语义准确：AI 理解上下文，按语义划分词组
- 覆盖率高：逐词对应，覆盖几乎所有文字
- 无需算法：省去复杂的中文分词逻辑

5.2 并行翻译控制

采用队列 + 并发控制模式，最大化翻译效率：

```
const queue = [...blocks];
const executing = [];
const concurrency = 10;

while (queue.length > 0 || executing.length > 0) {
  while (executing.length < concurrency && queue.length > 0) {
    const block = queue.shift();
    const promise = processBlock(block).finally(() => {
      executing.splice(executing.indexOf(promise), 1);
    });
    executing.push(promise);
  }
  if (executing.length > 0) {
    await Promise.race(executing);
  }
}
```

5.3 Safari 兼容性处理

Safari 扩展与 Chrome 存在重要差异，需要特别处理：

变量声明

Safari 内容脚本共享全局命名空间，必须使用 `var`：

```
// Chrome - 正常
const isSafari = typeof browser !== 'undefined';

// Safari - 必须使用 var
var isSafari = typeof browser !== 'undefined' && browser.runtime;
var _chrome = isSafari ? browser : chrome;
```

存储 API 适配

```
var storageAPI = {
    async get(keys) {
        if (isSafari) {
            return await browser.storage.local.get(keys);
        } else {
            return new Promise((resolve) => {
                chrome.storage.sync.get(keys, resolve);
            });
        }
    }
};
```

6. 跨平台适配

6.1 Chrome vs Safari 差异对比

特性	Chrome	Safari
Manifest 版本	V2/V3	V2
变量作用域	隔离	共享
存储 API	chrome.storage	browser.storage
加载方式	开发者模式	Xcode 构建

调试工具	DevTools	Safari	DevTools
代码签名	不需要	需要（免费账号可）	

6.2 代码共享策略

Chrome 和 Safari 共享 95% 以上的代码，通过条件编译和适配层实现跨平台：

文件同步流程

```
修改 chrome/ 目录
  |
  ▼
运行 sync-to-safari.sh
  |
  ▼
复制到 safari/AI Web Translator/Shared (Extension)/Resources/
  |
  ▼
Xcode 构建运行
```

平台检测

```
var isSafari = typeof browser !== 'undefined' && browser.runtime;
var _chrome = isSafari ? browser : chrome;
```

7. 部署与构建

7.1 Chrome 部署

开发者模式

1. 打开 chrome://extensions/
2. 开启「开发者模式」
3. 点击「加载已解压的扩展程序」
4. 选择 chrome/ 文件夹

发布到 Web Store

1. 打包扩展为 .zip
2. 登录 Chrome Web Store Developer Dashboard
3. 上传并提交审核

7.2 Safari 构建

环境要求

- macOS 12.0+
- Xcode 14.0+
- Apple Developer Account（免费即可）

构建步骤

1. 配置代码签名（Team ID）
2. 运行 ./quick-build.sh 或 Xcode 构建
3. 在 Safari 中启用扩展
4. 授予网站访问权限

一键构建脚本

```
#!/bin/bash
# quick-build.sh

# 1. 同步代码到 Safari 项目
./scripts/sync-to-safari.sh

# 2. Xcode 构建
xcodebuild -project safari/AI\ Web\ Translator/*.xcodeproj \
  -scheme "AI Web Translator (macOS)" \
  -destination "platform=macOS" \
  build

# 3. 运行应用
open safari/AI\ Web\ Translator/build/DerivedData/...
```

7.3 图标资源

类型	尺寸	用途
扩展图标	16x16, 48x48, 128x128	工具栏、商店
App 图标	16-1024 全尺寸	macOS App
Toolbar 图标	16, 19, 32, 38, 48, 72	Safari 工具栏

7.4 未来扩展

- 支持更多浏览器（Firefox、Edge）
- Manifest V3 升级
- 多语言支持（日、韩、德等）
- 离线翻译能力
- Mac App Store 上架